

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 8

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Dynamic Memory Allocation and Valgrind Leak Detection

Question:

Develop a program using `malloc()`, `calloc()`, `realloc()`, and `free()`. Monitor memory allocation behavior and identify memory leaks using Valgrind.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
    printf("=== Dynamic Memory Allocation Demo ===\n");

    /* malloc() */
    int *a = malloc(5 * sizeof(int));

    if (a == NULL)
    {
        perror("malloc");
        return 1;
    }

    printf("malloc() allocated 5 integers at %p\n", (void *)a);

    for (int i = 0; i < 5; i++)
        a[i] = i + 1;

    /* calloc() */
    int *b = calloc(5, sizeof(int));

    if (b == NULL)
    {
        perror("calloc");
        free(a);
        return 1;
    }

    printf("calloc() allocated 5 zero-initialized integers at %p\n",
        (void *)b);

    printf("calloc values: ");

    for (int i = 0; i < 5; i++)
        printf("%d ", b[i]);

    printf("\n");

    /* realloc() */
    int *temp = realloc(a, 10 * sizeof(int));
```

```
/* realloc() */
int *temp = realloc(a, 10 * sizeof(int));

if (temp == NULL)
{
    perror("realloc");
    free(a);
    free(b);
    return 1;
}

a = temp;

printf("realloc() resized malloc block to 10 integers at %p\n",
      (void *)a);

/* Separate allocation used to demonstrate a leak */
char *leak = malloc(64);

if (leak == NULL)
{
    perror("malloc");
    free(a);
    free(b);
    return 1;
}

strcpy(leak, "OSSP memory leak demonstration");

printf("Additional 64-byte block allocated at %p\n",
      (void *)leak);

free(a);
free(b);

printf("Main malloc/calloc blocks successfully freed\n");

if (argc > 1 && strcmp(argv[1], "clean") == 0)
{
    free(leak);
    printf("Clean mode: extra block also freed.\n");
}
else
{
    printf("Leak mode: extra 64-byte block intentionally leaked\n");
}

return 0;
}
```

```
/* realloc() */
int *temp = realloc(a, 10 * sizeof(int));

if (temp == NULL)
{
    perror("realloc");
    free(a);
    free(b);
    return 1;
}

a = temp;

printf("realloc() resized malloc block to 10 integers at %p\n",
      (void *)a);

/* Separate allocation used to demonstrate a leak */
char *leak = malloc(64);

if (leak == NULL)
{
    perror("malloc");
    free(a);
    free(b);
    return 1;
}
```

 Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots

Output:

Valgrind – Memory Leak Detection

```
haasini@SSH:~/OSSP_upload/Practical_Week8$ valgrind --leak-check=full --show-leak-kind
nds=all ./P8_T1
==1661== Memcheck, a memory error detector
==1661== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1661== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1661== Command: ./P8_T1
==1661==
=== Dynamic Memory Allocation Demo ===
malloc() allocated 5 integers at 0x4a74480
calloc() allocated 5 zero-initialized integers at 0x4a744e0
calloc values: 0 0 0 0 0
realloc() resized malloc block to 10 integers at 0x4a74540
Additional 64-byte block allocated at 0x4a745b0
Main malloc/calloc blocks successfully freed.
Leak mode: extra 64-byte block intentionally NOT freed.
==1661==
==1661== HEAP SUMMARY:
==1661==   in use at exit: 64 bytes in 1 blocks
==1661==   total heap usage: 5 allocs, 4 frees, 1,168 bytes allocated
==1661==
==1661== 64 bytes in 1 blocks are definitely lost in loss record 1 of 1
==1661==   at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-
linux.so)
==1661==   by 0x109413: main (P8_T1.c:60)
==1661==
==1661== LEAK SUMMARY:
==1661==   definitely lost: 64 bytes in 1 blocks
==1661==   indirectly lost: 0 bytes in 0 blocks
==1661==   possibly lost: 0 bytes in 0 blocks
==1661==   still reachable: 0 bytes in 0 blocks
==1661==   suppressed: 0 bytes in 0 blocks
==1661==
==1661== For lists of detected and suppressed errors, rerun with: -s
==1661== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Valgrind – Clean Execution

```
haasini@SSH:~/OSSP_upload/Practical_Week8$ valgrind --leak-check=full --show-leak-kind
nds=all ./P8_T1 clean
==1665== Memcheck, a memory error detector
==1665== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1665== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1665== Command: ./P8_T1 clean
==1665==
=== Dynamic Memory Allocation Demo ===
malloc() allocated 5 integers at 0x4a74480
calloc() allocated 5 zero-initialized integers at 0x4a744e0
calloc values: 0 0 0 0 0
realloc() resized malloc block to 10 integers at 0x4a74540
Additional 64-byte block allocated at 0x4a745b0
Main malloc/calloc blocks successfully freed.
Clean mode: extra block also freed.
==1665==
==1665== HEAP SUMMARY:
==1665==   in use at exit: 0 bytes in 0 blocks
==1665==   total heap usage: 5 allocs, 5 frees, 1,168 bytes allocated
==1665==
==1665== All heap blocks were freed -- no leaks are possible
==1665==
==1665== For lists of detected and suppressed errors, rerun with: -s
==1665== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Observations:

1. malloc() dynamically allocates uninitialized memory from the heap.
2. calloc() dynamically allocates memory and initializes all allocated bytes to zero.

3. `realloc()` changes the size of an existing allocation and may return a different address.
4. `free()` releases dynamically allocated memory back to the heap.
5. The program demonstrates intentional memory leak by allocating 64 bytes without freeing.
6. Valgrind detects memory that remains allocated when the program exits.
7. Leak execution reports 64 bytes in 1 blocks definitely lost.
8. Clean execution demonstrates that all allocated blocks were freed.
9. Valgrind provides comprehensive memory error detection and heap analysis.
10. Memory leak detection is critical for identifying resource wastage and potential bugs.

Task 2: Copy-on-Write Behavior After `fork()`

Question:

Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

Source Code:



haasini@SSH: ~/OSSP_upload



GNU nano 7.2

P8_T2.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define SIZE_MB 64
#define PAGE_SIZE 4096

void show_memory(const char *title)
{
    FILE *fp;
    char line[256];

    printf("\n--- %s ---\n", title);

    fp = fopen("/proc/self/smaps_rollup", "r");
    if (fp == NULL)
    {
        perror("fopen");
        return;
    }

    while (fgets(line, sizeof(line), fp))
    {
        if (strncmp(line, "Rss:", 4) == 0 ||
            strncmp(line, "Pss:", 4) == 0 ||
            strncmp(line, "Shared_Dirty:", 13) == 0 ||
            strncmp(line, "Private_Dirty:", 14) == 0)
        {
            printf("%s", line);
        }
    }

    fclose(fp);
}

int main(void)
{
    setbuf(stdout, NULL);

    size_t size = SIZE_MB * 1024 * 1024;

    char *data = malloc(size);

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    /*
     * Touch every page so physical pages are actually allocated.
     */
    for (size_t i = 0; i < size; i += PAGE_SIZE)
        data[i] = 1;

    printf("=== Copy-on-Write Demonstration ===\n");
}
```



haasini@SSH: ~/OSSP_upload



GNU nano 7.2

P8_T2.c *

```
    * Touch every page so physical pages are actually allocated.
    */
    for (size_t i = 0; i < size; i += PAGE_SIZE)
        data[i] = 1;

    printf("=== Copy-on-Write Demonstration ===\n");
    printf("Parent PID      : %d\n", getpid());
    printf("Allocated memory : %d MB\n", SIZE_MB);
    printf("Buffer address   : %p\n", (void *)data);
    printf("Initial value    : %d\n", data[0]);

    show_memory("Parent BEFORE fork");

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0)
    {
        printf("\n=== CHILD PROCESS ===\n");
        printf("Child PID      : %d\n", getpid());
        printf("Buffer address : %p\n", (void *)data);
        printf("Value before modification: %d\n", data[0]);

        show_memory("Child immediately AFTER fork");

        printf("\nChild modifying every memory page...\n");

        for (size_t i = 0; i < size; i += PAGE_SIZE)
            data[i] = 2;

        printf("Value after modification : %d\n", data[0]);

        show_memory("Child AFTER modification");

        free(data);

        return 0;
    }

    wait(NULL);

    printf("\n=== PARENT PROCESS ===\n");
    printf("Parent value after child modification: %d\n", data[0]);

    show_memory("Parent AFTER child exits");

    free(data);

    printf("\nCopy-on-Write demonstration completed.\n");

    return 0;
}
```

Output:

```
haasini@SSH:~/OSSP_upload/Practical_Week8$ ./P8_T2
=== Copy-on-Write Demonstration ===
Parent PID      : 1680
Allocated memory : 64 MB
Buffer address   : 0x7d8c183ff010
Initial value    : 1

--- Parent BEFORE fork ---
Rss:             67080 kB
Pss:             65723 kB
Shared_Dirty:    0 kB
Private_Dirty:   65644 kB

=== CHILD PROCESS ===
Child PID       : 1681
Buffer address  : 0x7d8c183ff010
Value before modification: 1

--- Child immediately AFTER fork ---
Rss:             66472 kB
Pss:             32882 kB
Shared_Dirty:    65600 kB
Private_Dirty:   40 kB

Child modifying every memory page...
Value after modification : 2

--- Child AFTER modification ---
Rss:             66472 kB
Pss:             65650 kB
Shared_Dirty:    64 kB
Private_Dirty:   65576 kB

=== PARENT PROCESS ===
Parent value after child modification: 1

--- Parent AFTER child exits ---
Rss:             67208 kB
Pss:             65730 kB
Shared_Dirty:    0 kB
Private_Dirty:   65644 kB

Copy-on-Write demonstration completed.
```

Observations:

1. `fork()` creates a child process with a separate virtual address space.
2. Linux initially avoids immediately copying all parent memory pages to the child.
3. Parent and child can initially reference shared physical pages through Copy-on-Write mechanism.
4. COW pages are copied when one process attempts to write to them.
5. The child modification causes affected memory pages to become private copies.
6. The parent's data remains unchanged (value 1) despite modification by the child (value 2).
7. The same virtual address (0x7d8c183ff010) appears in parent and child through COW mapping.
8. `/proc/self/smaps_rollup` provides detailed memory accounting information including RSS and PSS.
9. RSS (Resident Set Size) represents the actual resident memory; PSS proportionally accounts for shared memory.
10. `Private_Dirty` increases when COW produces modified private pages in the child process.